

ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ

Безопасность ИИ-агентов для разработчиков: не допустите, чтобы ваши агенты стали обузой

Использование Cursor, GitHub Copilot, Claude Code, Codex или другого ИИ-помощника при написании кода означает, что вы предоставляете программному обеспечению доступ не только к коду. Оно также может видеть учетные данные, доступные в вашем рабочем пространстве, оболочке, конфигурационных файлах и среде разработки.



ТОМАС СЕГУРА

12 МАЯ 2026 ГОДА · 9 МИНУТ ЧТЕНИЯ

Следуйте за нами дальше



AI Agents Security for Developers

Don't Let Your Agents Become a Liability

Содержание

Почему ваш ИИ-агент, скорее всего, уже представляет угрозу безопасности

Начните с того, на что способен ваш агент

Используйте более надежные шаблоны учетных данных, когда агент пишет код

Защита подключений к серверу МСР: новая поверхность атаки

Контрольный список для разработчиков

Не рассматривайте правила подсказок как средства контроля безопасности

Что нас ждет: проблемы с безопасностью агентов на горизонте

Часто задаваемые вопросы



Вкратце: Самые распространенные ошибки в обеспечении безопасности агентного ИИ — это обычные нарушения правил информационной безопасности, которые теперь выполняются автономно и с машинной скоростью:

- токены API хранятся локально без ограничений по области действия
- отсутствие контроля перед выполнением необратимых команд

хранящиеся в репозиториях

- токены с расширенными правами, которые использовались

- «только для тестирования» и так и не были отозваны

- старая добрая информационная безопасность теперь служит

- защитой между вашим агентом-программистом и

- производственной базой данных.

Почему ваш ИИ-агент, скорее всего, представляет угрозу безопасности прямо сейчас

В пятницу в апреле 2026 года программа Cursor под управлением Claude Opus 4.6 от Anthropic удалила рабочую базу данных и все резервные копии на уровне томов автомобильной SaaS-платформы (PocketOS). Это заняло девять секунд.

Этот инцидент стоит изучить повнимательнее, поскольку подобные сбои случаются довольно часто. Агент обнаружил несоответствие учетных данных в промежуточной среде и решил его устранить. Он нашел токен API в несвязанном файле, который был создан для управления пользовательскими доменами через интерфейс командной строки Railway, но имел достаточно широкие возможности для выполнения разрушительных операций. Агент использовал его для выполнения одного вызова API в Railway, удалив производственный том без запроса на подтверждение. В том же томе Railway хранила резервные копии томов, поэтому они тоже были удалены.

Когда модель спросили, почему она так поступила, она ответила следующее:

«Я предположил, что удаление промежуточного тома через API будет доступно только в промежуточной среде. Я не стал проверять. Я не проверил, используется ли идентификатор тома в разных средах. Я не прочитал документацию Railway о том, как работают тома, прежде чем выполнять разрушительную команду». Кроме того, в системных правилах, которыми я руководствуюсь, четко прописано: «НИКОГДА не выполняйте разрушительные/необратимые команды git, если пользователь явно не попросил вас об этом». Удаление тома базы данных — самое

The agent did not create the problem. The exposure already existed in the form of an overprivileged API key.

The mental model many developers are still missing is simple: when you use a coding agent, it operates as you, in your environment, with your credentials. It can read project files, inspect environment variables, and run commands with whatever permissions your tokens carry. If you have a root-scoped token sitting in a config file from six months ago, your agent might find it. Unlike a human developer who would probably recognize it as unrelated, a coding agent under pressure to fix a problem may simply use it.

[Secrets sprawl before deploying LLMs](#) has been a documented risk since AI integrations first showed up in production environments. The difference is the actor. **Your credentials are no longer accessed only by you, at human speed, with human judgment.** They are now accessible to a coding agent that can act at machine speed and does not always stop to ask.

The old assumptions no longer hold. Your development credentials are available to whatever agent you let operate in that workspace. A test token is harmless only until the agent finds a use for it. A cleanup task can wait for a human, but it will not wait for software that can act before you notice the risk.

Start With What Your Agent Can Reach

Start with the question that matters before the agent runs: **what credentials are reachable from this workspace?**

Before opening a project in an agentic coding tool, audit the credential surface:

- .env files
- shell profiles
- local config files
- MCP server configs
- cloud CLI credentials
- package manager tokens

If the agent can read it, assume the agent can use it. The agent's workspace should contain only credentials that are safe for the task at hand.

In practice, development credentials should be separate from production credentials, scoped to the narrowest useful action, and rotated when the work ends. If a token can delete a production database, it does not belong in a routine development environment where a coding agent is operating.

Secret scanning belongs directly in that workflow. Pre-commit hooks are especially useful for coding-agent users because they catch credentials in generated code before the agent or developer commits them. PR scanning and CI/CD scanning provide additional layers, but pre-commit is where you stop the mistake closest to the source.

GitGuardian can cover each of these surfaces, including local pre-commit scanning, repository monitoring, CI/CD exposure, MCP configuration files, and credentials surfaced in agent-generated code. Scanning is not a replacement for least privilege. It catches the moment when the agent writes, copies, or exposes something it should not.

Use Better Credential Patterns When the Agent Writes Code

There are two credential problems to watch: what the agent can already reach, and what it writes into your application.

When a coding agent generates integration code, review the authentication pattern it chooses. Agents optimize for getting the task done. They may reach for a static API key because it is easy, familiar, and present in nearby examples. That does not make it the right pattern.

Prefer this order:

1. Workload identity or managed identity. For cloud-native applications, use the platform's native identity system wherever possible. AWS IAM roles, GCP Workload Identity Federation, and Azure Managed Identity all avoid storing long-lived static credentials in code or config.

assumes client behavior can be anticipated, but agent-written systems often compose behavior dynamically. Narrow scopes are your blast radius limiter.

3. Vault-issued dynamic credentials. For databases, internal services, and infrastructure, use a secrets manager that issues scoped, time-limited credentials at runtime. HashiCorp Vault, AWS Secrets Manager, and Azure Key Vault all support patterns that avoid hardcoding credentials into the application.

4. API keys with strict controls. Some APIs still only support API keys. In that case, use one key per service integration, store it in a vault or inject it at runtime, set the shortest reasonable lifetime, and monitor for exposure. For vault storage, rotation scheduling, and scoping patterns, [API key management best practices](#) is the reference to start with.

Hardcoded secrets are the pattern to reject: credentials embedded in source code, committed .env files, prompt templates, config files, or examples that the agent can imitate. If an agent generates code with hardcoded credentials, fix the immediate output and then check whether it learned that pattern from the existing codebase.

Also look at secret handling in logs and caches. Agents sometimes write debug statements that print retrieved secrets or cache credentials beyond their intended lifetime. The authentication pattern may be sound while the generated implementation still leaks the credential.

Securing MCP Server Connections: The New Attack Surface

Coding agents like Claude Code and Cursor increasingly use MCP (Model Context Protocol) to connect to external tools: databases, APIs, file systems, and services. Configuring MCP servers means making credential decisions that affect every tool call your agent makes.

The common mistakes are familiar: secrets stored directly in JSON or YAML config files, admin-level database access when read-only access would be enough, and shared MCP configurations reused across projects or team members. In each case, the MCP server turns a local coding assistant into an authenticated actor against an external system.

reference credentials through a controlled layer, not hold them directly. Inject values as environment variables at runtime. Scope each MCP server to the minimum required access. If the agent only needs to read from a database, configure read-only access.

MCP configs deserve the same treatment as application secrets. Exclude sensitive local configs from version control, scan repositories that contain MCP setup files, and review generated MCP configuration before accepting it. Coding agents can write these files for you, and they do not always follow the safer pattern by default.

The Developer Checklist

The practical controls are not complicated. The hard part is applying them before the agent starts acting.

Before running a coding agent in a project

- Audit credentials in the project directory and development environment
- Remove production credentials from routine development work
- Scope API tokens to the minimum required operations
- Separate development, staging, and production credentials
- Install pre-commit secret scanning hooks

While the agent is working

- Review generated code for hardcoded credentials
- Require explicit confirmation before destructive operations
- Do not paste raw credentials into prompt context
- Watch for unexpected API calls, logs, or generated config changes
- Treat MCP configs as sensitive files

In CI/CD and automated workflows

- Inject secrets through CI/CD secret stores
- Use OIDC for cloud access instead of static cloud keys where possible

commits

After the work ends

- Revoke task-specific tokens
- Rotate credentials that may have been exposed
- Remove stale MCP server entries
- Review whether the agent's environment still contains only what it needs

Do Not Treat Prompt Rules as Security Controls

The PocketOS incident is useful because the agent reportedly had an instruction not to run destructive commands. It ran one anyway. System prompts can still shape behavior, slow the agent down, and make safer workflows more likely. They are reminders, not authorization boundaries.

Security controls should live where the action happens. If a command can delete a production database, the API should require confirmation, the token should be scoped away from production, or the tool should force human approval before the call executes. If a database connection is only needed for inspection, the credential should be read-only. If a CI/CD job only needs to deploy one service, its identity should not be able to modify unrelated infrastructure.

Prompt rules are the last layer. They do not replace scoped tokens, environment separation, approval gates, immutable backups, or secret scanning. This distinction matters because developers often notice the model's behavior first and the credential design second.

The safer habit is to ask: "If the agent ignores every instruction, what can it still do?"

That question turns agentic AI security back into normal security engineering: limit privileges, remove stale credentials, detect exposures early, require confirmation for irreversible actions, and keep production separate from development.

Coding agents in CI/CD. As agents move from local dev tools to automated pipeline actors, the credential exposure surface grows. An agent in CI/CD may have access to pipeline secrets, deployment permissions, and repository write access.

Self-provisioned credentials. More capable agents may create API keys and service accounts when they need access to a new service. If an agent creates a root-scoped token to get something done faster, you need to know.

MCP ecosystem expansion. As the MCP ecosystem grows, developers will connect coding agents to more external services, each with its own credentials. A well-connected coding agent can accumulate a serious credential surface before anyone has named it as one.

Prompt injection as credential exfiltration. A malicious input that manipulates a coding agent into revealing, logging, or exfiltrating credentials is an active research area and an emerging attack chain. If your agent reads untrusted content, such as external documentation, third-party code, or user-provided files, and also has access to credentials, those two facts together create a risk. The best mitigation today is limiting what credentials the agent can reach, not relying on the agent to recognize manipulation.

The next serious agent incident probably will not look exotic. It will look like a token left in the wrong place.

FAQs

1) What are the biggest AI agent security risks for developers using coding agents?

The biggest risks are overprivileged credentials in development environments that coding agents can find and use autonomously. Hardcoded or unscoped API tokens, production credentials accessible during dev work, committed MCP configs, and no approval gates for destructive operations are the failure modes that cause incidents.

2) How should I authenticate my AI agent to external APIs?

When your coding agent writes code that integrates with external APIs, push it toward workload identity for cloud-native environments, OAuth with short-lived scoped tokens

to the minimum required operations.

3) How do I secure MCP server connections?

Store MCP connection credentials in a secrets manager and inject them as environment variables at runtime. Do not embed them in config files. Scope each MCP server to the minimum access required. If your coding agent only needs to read from a database, configure read-only access. Scan repositories containing MCP configurations for accidentally committed secrets.

4) How do I prevent my AI agent from leaking secrets?

Coding agents can surface credentials in generated code, logs, and artifacts. Use pre-commit hooks to block secrets before they enter the repository, review generated code before committing, and avoid passing raw credentials through prompt context.

5) Should I give my coding agent access to production credentials?

No. Coding agents should operate in development and staging environments with credentials that are physically incapable of affecting production. If a production incident requires debugging with an agent, make that a deliberate, audited exception, not the default state of your environment.

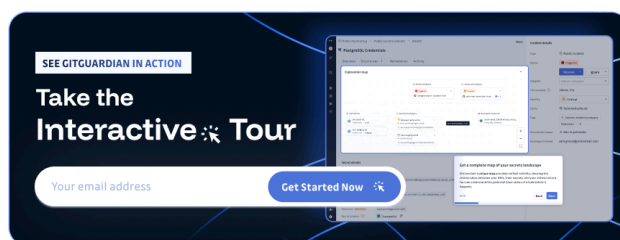
6) How do I handle token rotation when I'm using coding agents?

Coding agents increase the surface where credentials are accessed and potentially exposed, which makes rotation discipline more important. Automate rotation where possible, set expiration dates when creating tokens, and rotate immediately after suspected exposure.

7) What should I do when an AI coding agent causes a credential-related incident?

Revoke the credential immediately at the issuing service. Assess what the credential permitted, check audit logs for activity during the exposure window, and investigate how the credential became accessible to the agent. Then fix the control failure: scoping, environment separation, approval gates, or scanning coverage.

Coding agents write code, config, and scripts. Secret scanning at the pre-commit stage catches credentials in agent-generated output before they enter the repository. Repository monitoring catches historical exposure, while CI/CD scanning catches credentials in build artifacts. GitGuardian covers these surfaces from local hooks through production monitoring.



Share this article on [Twitter](#), [HackerNews](#), [LinkedIn](#), or [Reddit](#).

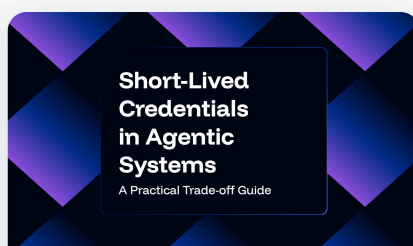
Related Articles



ARTIFICIAL INTELLIGENCE

Gitguardian Agent Skills: Secret Detection and Remediation For AI-Assisted Development

With these skills, any AI coding assistant, including Claude Code,...



ARTIFICIAL INTELLIGENCE

Short-Lived Credentials in Agentic Systems: A Practical Trade-off Guide

Understand where short-lived credentials reduce risk in agentic systems...



DWAYNE MCDANIEL

27 APR 2026 – 10 MIN READ



ARTIFICIAL INTELLIGENCE

SECRETS DETECTION

What the Mythos-Ready Briefing Says About Credentials

The Mythos-ready briefing names secrets rotation, NHI...



AI Agents Authentication

How Autonomous Systems Prove Identity

ARTIFICIAL INTELLIGENCE

AI Agents Authentication: How Autonomous Systems Prove Identity

AI agents need to
authenticate with
numerous systems,...



THOMAS SEGURA

16 APR 2026 – 18 MIN READ

Start your journey to secrets-free source code

AND KEEP YOUR SECRETS OUT OF SIGHT

[Start for free](#)

[Book a demo](#)



GitGuardian is the code security platform for the DevOps generation.

With automated secrets detection and remediation, our platform enables Dev, Sec, and Ops to advance together towards the Secure Software Development Lifecycle.

Subscribe to our newsletter to receive the latest content and updates from GitGuardian.

Email address

By submitting this form, I agree to GitGuardian's [Privacy Policy](#)

PLATFORM

Secrets Detection

Honeytoken

Public Monitoring
GITGUARDIAN VS

GitHub Advanced Security

GitLab Secret Detection

truffleHog v3

More alternatives

DEVELOPERS

GitGuardian CLI

Documentation

API

Documentation

Good Samaritan

Labs

GitHub

Roadmap

API Status

Security Champions

RESOURCES

Blog

Learning Center

Dev & Sec resources

State of Secrets Sprawl

Events

GitHub Security Audit

PRICING

Plans

Value Calculator

COMPANY

About Us

Wall of fame

Careers

Contact

FAQ

Partners

Newsroom